



WILLSIM

FICHE DE COURS · RESSOURCE PÉDAGOGIQUE GRATUITE

IA embarquée sur microcontrôleur (TinyML)

Faire tourner un réseau de neurones sur un ESP32 : acquisition des mesures d'un accéléromètre, entraînement d'un petit réseau dense, quantification en entiers 8 bits, puis inférence en temps réel sur la carte, mémoire et latence mesurées.

NIVEAU

Intermédiaire

DURÉE

8 h (4 séances de 2 h)

PRÉREQUIS

Cours « Initiation à l'IA pour l'ingénierie », bases d'Arduino / C++

OUTILS

ESP32, MPU-6050, Arduino IDE ou PlatformIO, TensorFlow/Keras, LiteRT for Microcontrollers

OBJECTIFS DE LA FICHE

À la fin de cette fiche, vous saurez :

- justifier une inférence embarquée plutôt que distante par trois critères : latence, confidentialité, consommation ;
- acquérir des mesures d'accélération à fréquence d'échantillonnage fixe (50 Hz) et vérifier cette fréquence ;
- entraîner un réseau dense de moins de 3 000 paramètres, le quantifier en int8 et contrôler que la précision perd au plus 2 points ;
- déployer le modèle sur ESP32, dimensionner la *tensor arena* et mesurer la latence d'inférence en microsecondes.

Déroulé indicatif : séance 1, sections 1 à 3 (enjeux, chaîne, acquisition) ; séance 2, section 4 (entraînement) ; séance 3, sections 5 et 6 (quantification, mémoire) ; séance 4, sections 7 et 8 (déploiement, latence, exercice guidé).

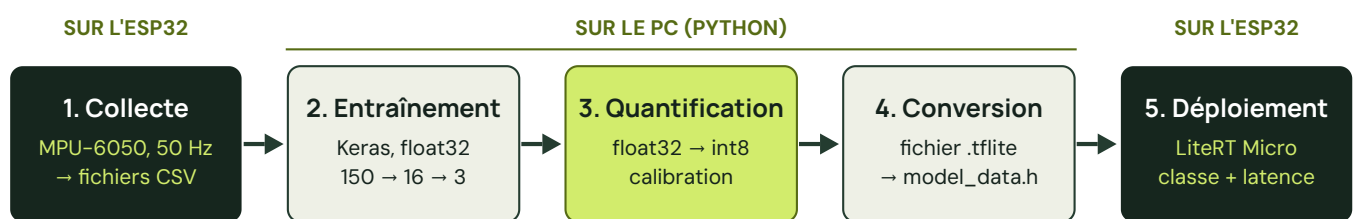
1. Pourquoi embarquer l'IA ?

Le **TinyML** (*tiny machine learning*) consiste à exécuter des modèles d'apprentissage automatique sur des microcontrôleurs : quelques centaines de kilo-octets de RAM, pas de système d'exploitation complet, de quelques milliwatts à quelques centaines de milliwatts. L'**entraînement** reste sur ordinateur ; la carte ne fait que l'**inférence**, c'est-à-dire la prédiction à partir de nouvelles mesures, avec un modèle figé. Fil rouge : reconnaître trois gestes (repos, secousse, cercle) avec un accéléromètre.

Inférence distante ou embarquée : les critères de choix

Critère	Inférence distante (serveur)	Inférence embarquée (microcontrôleur)
Latence	Aller-retour réseau, variable et sans garantie	Prévisible, limitée au calcul : bien moins d'une milliseconde pour un très petit réseau
Confidentialité	Les mesures brutes quittent l'appareil	Seule la décision sort (ex. : « secousse »), voire rien
Consommation	La radio domine : d'après la fiche technique de l'ESP32, l'émission Wi-Fi consomme typiquement de 180 à 240 mA	Calcul bref, la radio peut rester éteinte ; fonctionne aussi hors connexion

2. La chaîne complète, de la mesure à la carte



Le prétraitement (unités, ordre des axes, quantification de l'entrée) doit être identique aux étapes 2 et 5.

Figure 1 — Les cinq étapes de la chaîne TinyML et l'endroit où chacune s'exécute.

Le modèle converti, au format `.tflite` (sérialisé avec FlatBuffers), est compilé avec le croquis sous forme de tableau C. Sur la carte, l'interpréteur de **LiteRT for Microcontrollers** (anciennement TensorFlow Lite for Microcontrollers, ou TFLM) l'exécute sur chaque nouvelle fenêtre de mesures.

À RETENIR

Le microcontrôleur n'apprend rien : il exécute un modèle figé. Ajouter une classe ou changer de capteur impose de repasser par toute la chaîne. Et la première cause d'échec n'est pas le réseau, mais un prétraitement différent entre le PC et la carte.

3. Acquérir des données propres

3.1 Câblage

Le MPU-6050 (souvent monté sur un module GY-521) communique en I2C, à l'adresse 0x68 par défaut. Raccordement : **VCC** → **3V3**, **GND** → **GND**, **SDA** → **GPIO 21**, **SCL** → **GPIO 22**, broches du bus I2C par défaut (`Wire`) de la plupart des cartes ESP32 DevKit ; vérifiez le brochage de la vôtre.

ATTENTION · SÉCURITÉ

- Câblez **carte débranchée** et vérifiez VCC et GND avant de brancher l'USB.
- Alimentez le module en 3,3 V (broche 3V3) : l'ESP32 ne tolère pas un bus I2C à 5 V.
- Gestes : carte et capteur fixés sur un support rigide, câble tenu, voisins à distance.
- LiPo : chargeur protégé, ni court-circuit ni charge sans surveillance, aucune batterie gonflée.

3.2 Le croquis d'acquisition

Le croquis échantillonne à 50 Hz avec un **échancier fixe** : chaque échéance est calculée à partir de la précédente, et non de l'instant courant, ce qui évite toute dérive. Le filtre passe-bas numérique du MPU-6050 est réglé à 21 Hz, sous la fréquence de Nyquist (25 Hz), pour limiter le repliement de spectre. Les gestes de la main étant lents (quelques hertz), 50 Hz laisse une marge confortable.

acquisition.ino – bibliothèque Adafruit MPU6050 (gestionnaire de bibliothèques)

```
#include <Adafruit_MPU6050.h> // installe aussi Adafruit Unified Sensor et BusIO

const uint32_t kPeriodeUs = 20000; // 50 Hz
const float kG = 9.80665f; // m/s² -> g
Adafruit_MPU6050 mpu;
uint32_t echeance;

void setup() {
  Serial.begin(115200);
  if (!mpu.begin()) { // I2C 0x68 ; ESP32 : SDA = GPIO 21, SCL = GPIO 22
    Serial.println("MPU-6050 introuvable : vérifiez le câblage");
    while (true) delay(100);
  }
  mpu.setAccelerometerRange(MPU6050_RANGE_4_G);
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ); // passe-bas sous fs/2 = 25 Hz
  echeance = micros();
}

void loop() {
  while ((int32_t)(micros() - echeance) < 0) {} // attend l'échéance
  echeance += kPeriodeUs; // échancier fixe : pas de dérive
  sensors_event_t a, g, t;
  mpu.getEvent(&a, &g, &t); // accélérations en m/s²
  Serial.printf("%lu,%.3f,%.3f,%.3f\n", (unsigned long)micros(),
    a.acceleration.x / kG, a.acceleration.y / kG, a.acceleration.z / kG);
}
```

Ouvrez le moniteur série à 115 200 bauds, répétez un même geste et copiez les lignes dans un fichier par classe (`repos.csv` , `secousse.csv` , `cercle.csv`). La première colonne est un horodatage en microsecondes : l'écart moyen entre deux lignes doit valoir 20 000 µs, et un écart isolé nettement plus grand trahit une échéance manquée. Le transtypage en `(int32_t)` garde la comparaison juste quand `micros()` déborde, environ toutes les 71 minutes.

4. Entraîner un petit réseau dense

Un réseau dense (perceptron multicouche) attend un vecteur de taille fixe. On découpe donc le flux en fenêtres de 1 s : 50 échantillons × 3 axes, aplatis en un vecteur de 150 valeurs en g, dans l'ordre `ax`, `ay`, `az`, `ax...`, ordre à reproduire à l'identique sur la carte. Pour mettre au point la chaîne sans matériel, le script génère des fenêtres **synthétiques** qui imitent les trois gestes ; remplacez ensuite `fenetre()` par la lecture de vos fichiers CSV.

Le réseau défini ci-dessous compte 150 × 16 poids + 16 biais = 2 416 paramètres dans la couche cachée et 16 × 3 + 3 = 51 dans la couche de sortie, soit **2 467 paramètres** : environ 9,9 Ko en float32 et 2,5 Ko en int8 (poids sur 8 bits, biais sur 32 bits).

Sur ces données synthétiques, faciles par construction, le script ci-dessous atteint 100 % de précision sur les 200 fenêtres de test : cela valide la chaîne, pas le modèle. Seule l'évaluation sur vos propres enregistrements fait foi.

```

import numpy as np
import tensorflow as tf

FS, N = 50, 50 # 50 Hz, fenêtres de 1 s (50 échantillons)
rng = np.random.default_rng(0)
tf.keras.utils.set_random_seed(0) # résultats reproductibles

def fenetre(c):
    """Fenêtre synthétique (N, 3) en g ; à remplacer par vos enregistrements."""
    t = np.arange(N) / FS
    x = np.zeros((N, 3)) + rng.normal(0, 0.1, 3) # Légère inclinaison de la carte
    x[:, 2] += 1.0 # gravité sur l'axe z
    f, a, p = rng.uniform(1, 2), rng.uniform(0.3, 0.6), rng.uniform(0, 2 * np.pi)
    if c == 1: # secousse : 3,5 à 7 Hz selon x
        x[:, 0] += 2 * a * np.sin(2 * np.pi * 3.5 * f * t + p)
    elif c == 2: # cercle : x et y en quadrature
        x[:, 0] += a * np.cos(2 * np.pi * f * t + p)
        x[:, 1] += a * np.sin(2 * np.pi * f * t + p)
    return x + rng.normal(0, 0.05, (N, 3)) # bruit de mesure

y = rng.integers(0, 3, 1200) # 0 repos, 1 secousse, 2 cercle
X = np.stack([fenetre(c) for c in y]).reshape(len(y), -1).astype("float32")
X_app, y_app, X_test, y_test = X[:1000], y[:1000], X[1000:], y[1000:]

model = tf.keras.Sequential([
    tf.keras.Input(shape=(3 * N,)), # 150 entrées : ax, ay, az, ax...
    tf.keras.layers.Dense(16, activation="relu"),
    tf.keras.layers.Dense(3, activation="softmax"),
])
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.fit(X_app, y_app, epochs=40, batch_size=32, validation_split=0.2, verbose=0)
print("Précision float32 :", model.evaluate(X_test, y_test, verbose=0)[1])

```

5. Quantifier en int8 et convertir en tableau C

La quantification affine associe à chaque réel r un entier q sur 8 bits grâce à une **échelle** S (*scale*) et un **point zéro** Z (*zero point*) :

$$r \approx S \cdot (q - Z) \quad q = \text{clamp}\left(\text{round}\left(\frac{r}{S}\right) + Z, -128, 127\right)$$

Les poids sont quantifiés de façon symétrique ($Z = 0$) ; les activations, avec une échelle et un point zéro par tenseur, calibrés en faisant passer quelques centaines de fenêtres **représentatives** dans le réseau. C'est la **quantification post-entraînement** (PTQ) : pas de réentraînement, des poids 4 fois plus petits et des calculs en entiers, rapides sur microcontrôleur.

tinymml_gestes.py (2/3) – quantification post-entraînement et conversion

```

def donnees_representatives(): # calibre les plages d'activation
    for i in range(200):
        yield [X_app[i:i + 1]]

conv = tf.lite.TFLiteConverter.from_keras_model(model)
conv.optimizations = [tf.lite.Optimize.DEFAULT]
conv.representative_dataset = donnees_representatives
conv.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
conv.inference_input_type = tf.int8 # entrée et sortie en int8
conv.inference_output_type = tf.int8
# une seule échelle de poids par couche dense : voir l'encadré ci-dessous
conv._experimental_disable_per_channel_quantization_for_dense_layers = True
modele_int8 = conv.convert()

```

```

interp = tf.lite.Interpreter(model_content=model_int8) # vérification sur PC
interp.allocate_tensors()
e, s = interp.get_input_details()[0], interp.get_output_details()[0]
echelle, zero = e["quantization"] #  $r = S(q - Z)$ 
justes = 0
for xi, yi in zip(X_test, y_test):
    q = np.clip(np.round(xi / echelle) + zero, -128, 127).astype(np.int8)
    interp.set_tensor(e["index"], q[None, :])
    interp.invoke()
    justes += int(np.argmax(interp.get_tensor(s["index"])[0]) == yi)
print("Précision int8 :", justes / len(y_test), "| S =", echelle, "Z =", zero)

with open("model_data.h", "w", encoding="utf-8") as f: # comme « xxd -i »
    f.write("#pragma once\n// Modèle int8 (%d octets)\n" % len(model_int8))
    f.write("alignas(16) const unsigned char g_model[] = { %s };\n"
            % ",".join(str(b) for b in model_int8))

```

Avec TensorFlow 2.20, la précision int8 est identique à celle du float32 (100 %) et l'entrée est quantifiée avec $S \approx 0,011$ g et $Z = -4$. Le modèle pèse 4 496 octets : la description du graphe et de la quantification s'ajoute aux 2,5 Ko de poids. Il ne contient que trois opérations, deux **FULLY_CONNECTED** (ReLU fusionnée) et un **SOFTMAX**.

ATTENTION · UNE ÉCHELLE DE POIDS PAR COUCHE

Par défaut, TensorFlow 2.20 donne sa propre échelle de poids à chaque neurone d'une couche dense (quantification *par canal*). Or le noyau **FULLY_CONNECTED** de Chirale_TensorFlowLite 2.0.0 n'en lit qu'une, la première, sans aucun message d'erreur : compilé sur PC avec un tel modèle, il ne reproduit les sorties de l'interpréteur Python que pour 24 des 200 fenêtres de test, et la précision tombe à 81 %. L'option expérimentale du script impose une échelle par couche : les sorties redeviennent identiques (200 sur 200).

TensorFlow 2.20 signale que `tf.lite.Interpreter` est déprécié au profit du paquet `ai-edge-litert` (Python 3.10 ou plus) : `from ai_edge_litert.interpreter import Interpreter`, mêmes méthodes.

6. Contraintes mémoire : flash, RAM et tensor arena

La **flash**, non volatile, contient le programme et les constantes ; la **RAM** (SRAM), volatile et rare, contient les variables. L'ESP32 classique offre 520 Ko de SRAM interne, dont une partie est prise par le système et les piles radio, et une flash externe, souvent de 4 Mo sur les cartes de développement.

La **tensor arena** est une zone de RAM réservée une fois pour toutes : l'interpréteur y range les tenseurs d'entrée, de sortie et intermédiaires ainsi que ses propres structures, sans aucune allocation dynamique pendant l'inférence. Sa taille ne se calcule pas simplement à l'avance : partez d'une valeur large (8 Ko ici), lisez `arena_used_bytes()` après `AllocateTensors()`, puis ajustez avec une marge. Trop petite, elle fait échouer `AllocateTensors()` ; trop grande, elle gaspille de la RAM.

À RETENIR

- Déclarez le modèle `const` : il reste en flash. Sans `const`, il serait recopié en RAM au démarrage.
- `MicroMutableOpResolver` n'enregistre que les opérations du modèle : le code des autres n'est pas lié, ce qui économise de la flash.
- Mesurez au lieu de deviner : le script Python donne la taille du modèle (4 496 octets ici), le rapport de compilation celle du programme et des variables globales, `arena_used_bytes()` celle de l'arena.

7. Boucle d'inférence et mesure de la latence

Le croquis final reprend l'acquisition, quantifie l'entrée **exactement comme en Python**, appelle `Invoke()` et le chronomètre. Sous Arduino IDE, installez la bibliothèque **Chirale_TensorFlowLite**, portage de LiteRT for Microcontrollers pour cartes mbed et ESP32 (testé par son auteur sur Arduino Nano ESP32). Sous ESP-IDF (ou PlatformIO avec ce framework), le composant **esp-tflite-micro** d'Espressif offre la même API, avec des noyaux optimisés (ESP-NN).

inference.ino (1/2) – model_data.h placé dans le dossier du croquis

```
#include <Chirale_TensorFlowLite.h>           // à inclure en premier
#include "tensorflow/lite/micro/micro_interpreter.h"
#include "tensorflow/lite/micro/micro_mutable_op_resolver.h"
#include "tensorflow/lite/schema/schema_generated.h"
#include <Adafruit_MPU6050.h>
#include "model_data.h"                       // g_model, écrit par Le script Python

constexpr int kFenetre = 50, kAxes = 3;      // 1 s à 50 Hz : 150 entrées
constexpr int kArena = 8 * 1024;             // à ajuster (voir arena_used_bytes)
alignas(16) static uint8_t arena[kArena];
const char* kClasses[] = {"repos", "secousse", "cercle"};
Adafruit_MPU6050 mpu;
tflite::MicroInterpreter* interp = nullptr;

void arret(const char* message) { Serial.println(message); while (true) delay(100); }

void setup() {
  Serial.begin(115200);
  if (!mpu.begin()) arret("MPU-6050 introuvable");
  mpu.setAccelerometerRange(MPU6050_RANGE_4_G);
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
  const tflite::Model* modele = tflite::GetModel(g_model);
  if (modele->version() != TFLITE_SCHEMA_VERSION) arret("Schéma incompatible");
  static tflite::MicroMutableOpResolver<2> ops; // seules les opérations du modèle
  ops.AddFullyConnected();
  ops.AddSoftmax();
  static tflite::MicroInterpreter mi(modele, ops, arena, kArena);
  interp = &mi;
  if (interp->AllocateTensors() != kTfLiteOk) arret("Arena trop petite");
  Serial.printf("Arena utilisée : %u octets\n", (unsigned)interp->arena_used_bytes());
}
```

inference.ino (2/2) – acquisition d'une fenêtre, inférence, chronométrage

```
void loop() {
  TfLiteTensor* entree = interp->input(0);
  const float echelle = entree->params.scale; // S et Z fixés à la conversion
  const int zero = entree->params.zero_point;
  uint32_t echeance = micros();
  for (int i = 0; i < kFenetre; i++) {
    while ((int32_t)(micros() - echeance) < 0) {}
    echeance += 20000; // 50 Hz
    sensors_event_t a, g, t;
    mpu.getEvent(&a, &g, &t);
    const float v[kAxes] = {a.acceleration.x, a.acceleration.y, a.acceleration.z};
    for (int k = 0; k < kAxes; k++) { // même quantification qu'en Python
      int q = (int)lroundf(v[k] / 9.80665f / echelle) + zero;
      entree->data.int8[i * kAxes + k] = (int8_t)constrain(q, -128, 127);
    }
  }
  const uint32_t t0 = micros();
  if (interp->Invoke() != kTfLiteOk) arret("Échec de l'inférence");
  const uint32_t duree = micros() - t0;
  const int8_t* sortie = interp->output(0)->data.int8;
  int c = 0;
  for (int j = 1; j < 3; j++) if (sortie[j] > sortie[c]) c = j;
  Serial.printf("%s (inférence : %lu µs)\n", kClasses[c], (unsigned long)duree);
}
```

La sortie est elle aussi en int8 : la quantification étant croissante, la classe la plus probable est l'indice du plus grand entier. Pour une probabilité, déquantifiez : $p = S(q - Z)$, ici $S = 1/256$ et $Z = -128$, valeurs fixées par le convertisseur pour un softmax. Avec 2 448 multiplications-accumulations, l'inférence est très courte devant la seconde d'acquisition : mesurez la **moyenne sur 100 inférences**, une mesure isolée étant trompeuse (cache de la flash, interruptions). Pour réagir plus vite, faites glisser la fenêtre par pas de 0,5 s.

Croquis compilés avec le cœur Arduino ESP32 3.3.12, Chirale_TensorFlowLite 2.0.0 et Adafruit MPU6050 2.2.9 (inférence : 349 196 octets de flash). Une version plus ancienne peut exiger un `ErrorReporter` en argument de `MicroInterpreter`.

8. Limites et bonnes pratiques

- **Pas de fuite** : séparez apprentissage et test par séance ou par personne, jamais au hasard.
- **Prévoir le rejet** : un softmax choisit toujours une classe ; ajoutez une classe « autre ».
- **Calibrer sur de vraies mesures** : ici, l'entrée couvre -1,37 à +1,45 g ; au-delà, elle sature.
- **Comparer carte et PC** : une même fenêtre doit donner les mêmes sorties int8 des deux côtés.
- **Opération manquante** : l'initialisation échoue ; ajoutez-la au résolveur (`Add...()`).

EXERCICE GUIDÉ · TROIS GESTES, DU CAPTEUR À LA DÉCISION

1. Enregistrez 60 fenêtres de 1 s par geste (apprentissage), puis 20 lors d'une autre séance (test).
2. Remplacez `fenetre()` par la lecture de vos CSV ; entraînez, quantifiez, exportez `model_data.h`.
3. Déployez `inference.ino`, réglez `kArena` à `arena_used_bytes()` + 20 % et vérifiez l'allocation.
4. Mesurez la latence moyenne d' `Invoke()` sur 100 inférences et la taille du programme.

Critères de réussite : précision int8 ≥ 90 % sur le jeu de test ; écart float32/int8 ≤ 2 points ; arena ajustée et notée ; latence, tailles du modèle et du programme consignées en une demi-page.

Indice : précision décevante ? Vérifiez d'abord le prétraitement (unités, ordre des axes, orientation), puis variez les enregistrements avant d'ajouter des neurones. Écart float32/int8 trop grand : plus de données représentatives.

Pour aller plus loin

- [LiteRT for Microcontrollers \(Google AI Edge\)](https://developers.google.com/edge/litert/microcontrollers/overview) — <https://developers.google.com/edge/litert/microcontrollers/overview>
- [Quantification entière post-entraînement](https://developers.google.com/edge/litert/conversion/tensorflow/quantization/post_training_integer_quant) — https://developers.google.com/edge/litert/conversion/tensorflow/quantization/post_training_integer_quant
- [Dépôt officiel tflite-micro](https://github.com/tensorflow/tflite-micro) — <https://github.com/tensorflow/tflite-micro>
- [Composant esp-tflite-micro \(Espressif\)](https://github.com/espressif/esp-tflite-micro) — <https://github.com/espressif/esp-tflite-micro>
- [R. David et al., TensorFlow Lite Micro \(article de référence\)](https://arxiv.org/abs/2010.08678) — <https://arxiv.org/abs/2010.08678>
- [Page du cours sur willsim.fr](https://willsim.fr/cours/ia-embarquee-tinymml.html) — <https://willsim.fr/cours/ia-embarquee-tinymml.html>

Fiche rédigée par l'équipe pédagogique WILLSIM, association loi 1901 (RNA W842013616) qui promeut l'usage de l'intelligence artificielle dans l'ingénierie et la mécatronique. Version 1.0 – octobre 2026.

© 2026 WILLSIM — Licence Creative Commons BY-NC-SA 4.0 (creativecommons.org/licenses/by-nc-sa/4.0/deed.fr) : vous pouvez copier, diffuser et adapter cette fiche à des fins non commerciales, en citant WILLSIM et en partageant vos adaptations sous la même licence.

Une erreur, une suggestion ? Écrivez à contact@willsim.fr · willsim.fr